

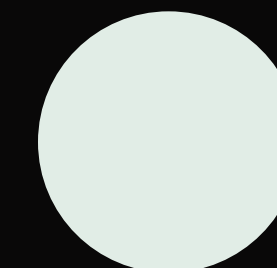
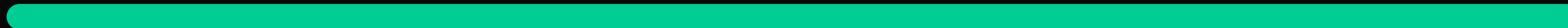
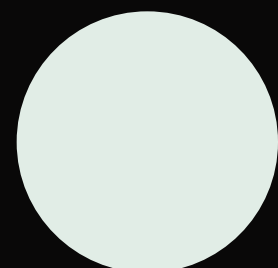


UNIVERSIDADE DE FORTALEZA - UNIFOR
DISCIPLINA: [T290] RESOLUÇÃO DE PROBLEMAS COM GRAFOS
PROF. DOUTORANDO RICARDO KASSNER CARUBBI

{Árvore Geradora Mínima}

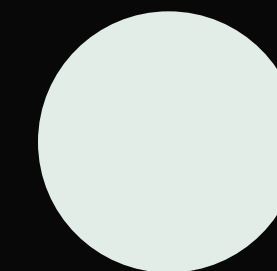
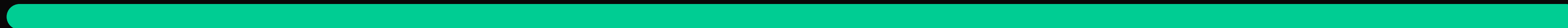
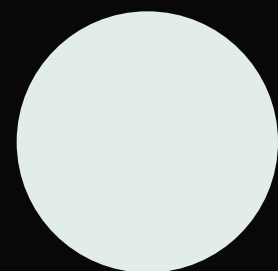
Grupo G

Maio, 2026



{Contextualização}

UVA 11228



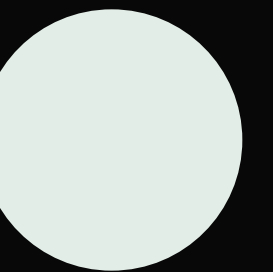
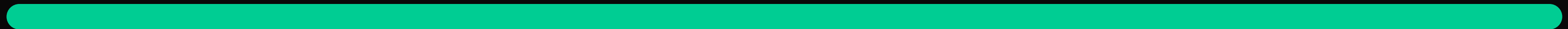
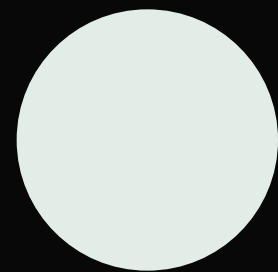
Contextualização

O problema 11228 – Transportation System apresenta um país fictício de Graphland. O país possui dezenas de cidades espalhadas por um plano cartesiano, mas não possui nenhuma via de ligação entre elas. O objetivo é construir um sistema de transporte que conecte todas as cidades, garantindo que seja possível viajar de qualquer ponto a qualquer outro, gastando o mínimo possível. Em termos de grafos, isso é a clássica busca por uma Árvore Geradora Mínima (MST).

No entanto, o problema introduz uma restrição: a regra do limiar de distância (R). O governo estabeleceu que cidades que estão próximas umas das outras (distância menor ou igual a R) pertencem ao mesmo "estado" e devem ser conectadas por estradas. Por outro lado, cidades mais distantes (distância maior que R) pertencem a estados diferentes, e a conexão entre estados deve ser feita exclusivamente por ferrovias

{Algoritmo Escolhido}

Kruskal



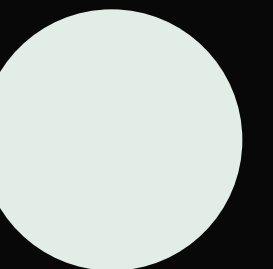
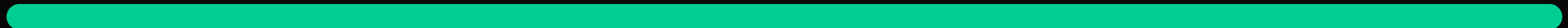
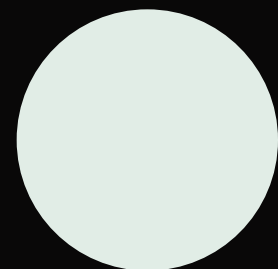
Algoritmo Escolhido

Para traduzir essa situação para o código, primeiro modelamos o mapa do país como um Grafo Completo. Cada cidade é um vértice, e calculamos a distância euclidiana (a hipotenusa) entre todos os pares de cidades possíveis, transformando essas distâncias nas arestas do grafo.

Para encontrar a solução de menor custo, foi optado o uso do Algoritmo de Kruskal. O Kruskal é um algoritmo guloso (greedy) clássico para encontrar Árvore Geradora Mínima. A premissa dele é simples: ele analisa as conexões disponíveis sempre da mais barata para a mais cara.

Como o algoritmo de Kruskal não funciona sozinho de forma eficiente, ele foi implementado em conjunto com a estrutura de dados Union-Find (ou Conjuntos Disjuntos). O papel do Union-Find na solução é ser o "gerente" das obras: antes de uma estrada ou ferrovia ser construída, perguntamos ao Union-Find se elas já estão conectadas por outro caminho. Se já estiverem, essa construção é ignorada para não criar ciclos fechados.

{Por que Kruskal?}

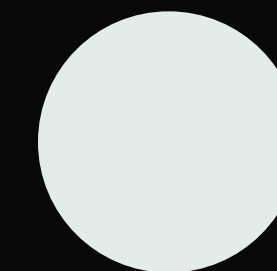
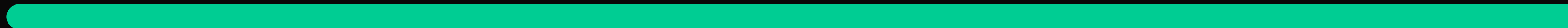
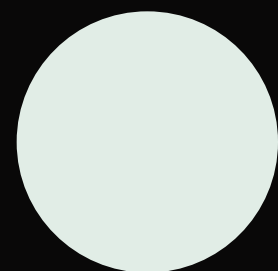


Por que Kruskal?

Como o Kruskal ordena todas as arestas do menor para o maior peso, ele naturalmente vai agrupar as cidades mais próximas primeiro. Isso significa que todas as estradas (que possuem distância $\leq R$) serão processadas antes das ferrovias.

Essa característica nos permite identificar os estados de forma dinâmica. Quando o algoritmo esgotar todas as conexões menores ou iguais a R , sabemos que os estados estão perfeitamente delineados. A partir desse ponto, qualquer nova aresta adicionada à árvore será, obrigatoriamente, uma ferrovia interligando esses estados.

{A Solução em Si}



A Solução em Si

Começamos lendo as coordenadas e calculando todas as distâncias possíveis. Nesse momento inicial, assume-se que não há conexões, ou seja, se temos 1000 cidades, temos inicialmente 1000 "estados" independentes.

Com as arestas ordenadas, iteramos sobre elas. Para cada conexão válida (que não forma ciclo segundo o Union-Find), é feita uma verificação simples:

- A distância é menor ou igual a R ? Se sim, estamos construindo uma estrada. Adicionamos esse valor ao custo das estradas. Além disso, como acabamos de unir dois polos isolados, o número total de estados no país diminui em 1.
- A distância é maior que R ? Se sim, estamos construindo uma ferrovia. Adicionamos esse valor ao custo das ferrovias. Neste ponto, o número de estados não diminui mais, pois estamos apenas ligando estados que já estavam consolidados.

Ao final do processamento de todas as cidades, teremos exatamente o número final de estados e os custos exatos. Por fim, aplicamos o arredondamento matemático exigido pelo enunciado (`Math.round()` no Java) e imprimimos a resposta no formato esperado pelo juiz online.

Obrigado!

